UniCampus ERP

College Enterprise Resource Planning System

SOFTWARE REQUIREMENTS SPECIFICATION

Version 1.0 • June 2026 • DRAFT

Field	Value
Document Title	UniCampus ERP SRS
Version	1.0 (Draft)
Date	June 23, 2026
Author	Zishan Ahmad
Institution	United Institute of Management (FUGS)
Audience	Principal, Faculty, IT Department, Developers
Status	For Review & Approval

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for UniCampus ERP — a full-stack, web-based College Enterprise Resource Planning system designed to digitize and streamline all administrative and academic operations of the United Group of Institutions (FUGS).

The primary goal is to replace the existing iCampus ERP (Digital Dreams Systems) with a custom-built, institution-owned solution that is open to future modification, fully auditable, and tailored to the specific workflows of the college.

1.2 Scope

UniCampus ERP will serve as the single digital platform for:

- Student lifecycle management (admission to graduation)
- Faculty and staff administration
- Academic scheduling (timetables, substitutions)
- Fee collection and financial reconciliation
- Library management with fine automation
- Hostel and bus registration with integrated payments
- Semester registration with date-controlled windows
- Attendance tracking with real-time dashboards
- Examination and marks management
- Placement and corporate relations
- Grievance and notice management
- Comprehensive audit and reporting

The system is NOT in scope for: LMS (lecture content delivery), video conferencing, payroll processing, or procurement management in Phase 1.

1.3 Definitions and Acronyms

Term / Acronym	Definition
ERP	Enterprise Resource Planning
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
RLS	Row-Level Security (PostgreSQL feature)
API	Application Programming Interface
JWT	JSON Web Token (for authentication)
OTP	One-Time Password
UPI	Unified Payments Interface (India)
HOD	Head of Department
CMS	Content Management System (for notices)
FUGS	Faculty of United Group of Institutions
MIS	Management Information System

Term / Acronym	Definition
UI/UX	User Interface / User Experience
SLA	Service Level Agreement

1.4 References

- Existing iCampus ERP (student.icampuserp.in) — reference screenshots provided
- PostgreSQL 14+ official documentation
- OWASP Top 10 Web Application Security Risks
- UGC Regulations on Online Examinations and Academic Records
- RBI Payment Aggregator Guidelines (for fee payment integration)
- Zishan Ahmad’s existing database schema (00–09 SQL scripts)

1.5 Overview of Document

The remainder of this document is organized as follows: Section 2 covers the overall product description and stakeholder overview. Section 3 provides detailed functional requirements for each module. Section 4 specifies non-functional requirements. Section 5 addresses system constraints. Section 6 defines the data model extension requirements. Section 7 covers UI/UX guidelines. Section 8 provides the implementation roadmap.

2. Overall Description

2.1 Product Perspective

UniCampus ERP is a greenfield, cloud-deployable web application. It will interface with:

- PostgreSQL database (existing schema as foundation)
- Payment gateway (Razorpay / PayU / CCAvenue) for fee collection
- SMS gateway (Fast2SMS / MSG91) for OTP and notifications
- Email SMTP for automated communications
- College bank account for settlement reconciliation
- Government APIs (DigiLocker optional in Phase 2)

2.2 User Roles and Access Matrix

The system defines eight primary user roles, each with distinct access boundaries:

Role	Users	Key Responsibilities	Access Level
Super Admin	1 (IT Head / Developer)	Full system configuration, DB access, user provisioning, role management	Unrestricted
Principal / Director	1–2	View MIS reports, approve policies, override decisions, view all modules	Read-all + approve
Account Officer	2–5	Fee collection, payment reconciliation, fine management, financial reports	Finance modules
Examination Controller	1–2	Marks entry control, exam schedule, result publishing, grade management	Exam + result modules
Librarian	1–3	Book catalog, issue/return, fine calculation, library membership	Library module
HOD (Dept Head)	1 per dept	Dept timetables, teacher assignments, course management, dept reports	Dept-scoped all
Faculty / Teacher	Many	Mark attendance, upload assignments, view timetable, apply for leave	Own data + batch
Administrative Staff	5–10	Hostel/bus registration processing, notices, student records	Admin modules
Student	All students	View timetable, attendance, marks, pay fees, register for hostel/bus/semester	Own data only
Guest / Public	Visitors	View public notices, contact info, admission info	Public pages only

2.3 General Constraints

- Must be accessible via modern web browsers (Chrome 90+, Firefox 90+, Safari 14+, Edge 90+) without any plugin installation
- Mobile-responsive design mandatory — 70% of students access portals from smartphones
- All financial transactions must go through a PCI-DSS compliant payment gateway
- Passwords must be hashed using bcrypt (min work factor 12) or Argon2id
- System must support concurrent access by at least 500 simultaneous users without degradation
- Deployment must support HTTPS only; HTTP redirects to HTTPS
- All date-sensitive operations (registration windows, fine timers) must use server-side timestamps, never client-side

- Must comply with India’s IT Act 2000 and relevant UGC data retention guidelines

2.4 Assumptions and Dependencies

- The college has an active internet connection with a static IP or domain for server hosting
- A Razorpay / PayU merchant account has been or will be created for the institution
- The existing PostgreSQL schema (files 01–09) will be extended, not discarded
- SMS credits will be purchased via the selected SMS gateway provider
- The principal’s office will designate a system administrator before go-live

3. Functional Requirements

Each module below is described with its actors, detailed requirements, and business rules. Requirements are tagged FR-MOD-NNN for traceability in future bug reports and changelogs.

3.1 Authentication and Session Management

3.1.1 Login

- FR-AUTH-001: Users shall log in with username/email + password. The system shall not indicate which field is wrong (generic error: “Invalid credentials”).
- FR-AUTH-002: After 5 consecutive failed login attempts, the account shall be locked for 15 minutes. The account officer or admin can manually unlock.
- FR-AUTH-003: Sessions shall expire after 30 minutes of inactivity. A 5-minute warning popup must appear before expiry.
- FR-AUTH-004: JWT tokens shall be used for API authentication with a 1-hour access token and 7-day refresh token stored in HttpOnly cookies.
- FR-AUTH-005: The system shall support OTP-based password reset via registered mobile number or email.
- FR-AUTH-006: All login events (success, failure, logout) shall be recorded in the audit.access_logs table with IP address, user agent, and timestamp.
- FR-AUTH-007: First-time login must force a password change from the default password.
- FR-AUTH-008: Admin can impersonate any user role for debugging, with all such sessions flagged in audit logs as impersonation sessions.

3.1.2 Role and Permission Management

- FR-AUTH-009: The RBAC system shall enforce permissions at both API gateway and database (RLS) levels — dual-layer enforcement.
- FR-AUTH-010: Super Admin can create, modify, and deactivate any user account. Deactivated accounts retain all data.
- FR-AUTH-011: Permission grants are audited. Any change to role_permissions triggers an audit log entry.

3.2 Student Module

3.2.1 Student Registration and Profile

- FR-STU-001: New students are registered by administrative staff or imported via CSV/Excel bulk upload with field validation.
- FR-STU-002: Each student profile contains: roll number, full name, date of birth, gender, contact, parent/guardian details, photo, department, batch, section, admission year, admission type (direct/counselling), category (General/OBC/SC/ST), and current status (active/discontinued/passed out).
- FR-STU-003: Students can update only: mobile number, profile photo, emergency contact, address. All other fields require staff/admin approval.
- FR-STU-004: Students can download their ID card, admit card, and bonafide certificate as PDF (auto-generated from stored data).
- FR-STU-005: Bulk operations (promote all students to next semester, mark graduated students) are available to admin/HOD.

3.2.2 Semester Registration

- FR-STU-010: Semester registration is only available within a configurable date window set by the admin (opening date + closing date + time, with timezone).

- FR-STU-011: The system shall block registration if: (a) pending fee dues exceed a configurable threshold, (b) library fines are unpaid above a threshold, or (c) previous semester result is withheld.
- FR-STU-012: Registration confirmation generates a PDF receipt with registration number, academic year, semester, subjects opted, and timestamp.
- FR-STU-013: Admin can extend the registration window or grant individual student exceptions with a reason recorded in the audit log.
- FR-STU-014: Students can view registration status: pending, submitted, approved, rejected. HOD approves registrations for their department.

3.3 Fee Management Module

3.3.1 Fee Structure

- FR-FEE-001: Admin/Account Officer defines fee heads (tuition, exam, library, hostel, bus, development) per department, semester, and academic year.
- FR-FEE-002: Fee concessions (scholarship, category-based, sibling discount) are configurable as percentage or flat amount.
- FR-FEE-003: Late payment fine is automatically calculated based on a configurable daily/weekly rate after the due date. Fine accumulates until payment.
- FR-FEE-004: Fee dues are visible to students in a breakdown format: original amount, concession, amount paid, balance, fine accrued to date.

3.3.2 Payment Processing

- FR-FEE-010: Online payment integrates with Razorpay (primary) supporting UPI, debit/credit cards, net banking, and wallets.
- FR-FEE-011: Every payment creates a transaction record with: payment_id, order_id, gateway_transaction_id, amount, timestamp, status, payment method, and student_id.
- FR-FEE-012: Failed transactions are logged and the student is shown a clear error with a retry option. No double-debit scenarios.
- FR-FEE-013: Successful payment triggers: (a) instant on-screen receipt, (b) PDF receipt generation for download, (c) SMS confirmation, (d) email with attached receipt.
- FR-FEE-014: Account Officer can record offline payments (cash/DD/NEFT) with manual receipt number and bank reference.
- FR-FEE-015: Daily collection report, ledger export (PDF + Excel), and reconciliation summary are available to Account Officer and Principal.
- FR-FEE-016: Refunds are initiated by Account Officer, recorded with reason, and tracked until settlement confirmation from the gateway.

3.3.3 Hostel Registration

- FR-FEE-020: Hostel registration is date-window controlled (opening/closing datetime configurable by admin).
- FR-FEE-021: Students select hostel category (AC/Non-AC, single/double/triple sharing), which determines the fee amount dynamically.
- FR-FEE-022: Room allotment is done by hostel warden (admin role variant) after payment confirmation on first-come-first-served basis.
- FR-FEE-023: Local guardian details are mandatory fields for hostel registration. Validation ensures contact number is a valid Indian mobile number.
- FR-FEE-024: Hostel registration status is reflected on the student’s dashboard with room number once allotted.

3.3.4 Bus Registration

- FR-FEE-030: Bus routes are configured by admin with stops, distances, and per-term fee.

- FR-FEE-031: Students select their nearest stop; fee is computed based on the stop. Payment integrates with the same payment gateway.
- FR-FEE-032: Bus pass is downloadable as a PDF/image after payment with QR code for verification.

3.4 Library Management Module

3.4.1 Catalog and Inventory

- FR-LIB-001: Librarian can add books with: ISBN, title, author(s), publisher, edition, year, category, total copies, available copies, and rack location.
- FR-LIB-002: Book search is available to all authenticated users by title, author, ISBN, or category with real-time availability status.
- FR-LIB-003: Physical inventory audit: librarian can mark books as lost, damaged, or archived.

3.4.2 Issue and Return

- FR-LIB-010: Issue requires: student lookup by roll number, barcode scan or book ID, and confirms no outstanding fines blocking new issues (configurable threshold).
- FR-LIB-011: Due date is calculated as: issue date + configurable loan period (default 14 days). Configurable per book category (reference books: in-library use only).
- FR-LIB-012: Return marks book available, calculates any fine if returned late.
- FR-LIB-013: Fine is Rs. [configurable] per day per book after due date. Fine is auto-added to the student's library fine balance.
- FR-LIB-014: Students can view: currently issued books, due dates, fine balance, and issue history.
- FR-LIB-015: Automated reminders (SMS + in-app notification) are sent 2 days before due date and on the due date.
- FR-LIB-016: Library fine balance blocks: (a) new book issues above threshold, (b) semester registration (if unpaid above threshold), (c) NOC issuance.
- FR-LIB-017: Fine payment can be done online (via payment gateway) or offline (at counter). Librarian records offline payments.

3.5 Timetable and Attendance Module

3.5.1 Timetable Management

- FR-TT-001: HOD creates timetables per batch/section/semester. Conflict detection prevents: teacher double-booking, classroom double-booking, batch double-booking.
- FR-TT-002: Timetable is viewable by students (their own batch), teachers (their own schedule), and HOD (all batches in department).
- FR-TT-003: Substitution workflow: faculty submits absence request → HOD assigns substitute → substitute receives notification → students see updated timetable for that day.
- FR-TT-004: Timetable export as PDF (print-ready weekly grid) for faculty and students.

3.5.2 Attendance Management

- FR-ATT-001: Faculty marks attendance per class session. Attendance can be marked as: Present, Absent, Medical Leave, Duty Leave.
- FR-ATT-002: Attendance percentage is calculated in real-time. Students at or below 75% are flagged with a visual alert on their dashboard.
- FR-ATT-003: Attendance below 60% triggers an automated notice to parents (SMS) and blocks exam eligibility (configurable threshold).
- FR-ATT-004: HOD can view and export attendance reports by: student, batch, subject, date range.

- FR-ATT-005: Faculty can edit attendance within 24 hours of marking with a reason. After 24 hours, only HOD can edit with mandatory audit log entry.
- FR-ATT-006: Students can view their subject-wise attendance with class count, present count, absent count, and percentage.
- FR-ATT-007: Condonation requests (for medical leave with proof) submitted by students are reviewed by HOD with approve/reject workflow.

3.6 Examination and Results Module

3.6.1 Marks Entry

- FR-EX-001: Exam Controller creates exam schedule (internal/external) with subject, date, time, max marks, and eligibility rules.
- FR-EX-002: Faculty enters internal assessment marks (sessional, assignments, viva) within a configurable window. After the window closes, entries are locked and require Exam Controller approval to modify.
- FR-EX-003: Marks are entered for each component (CA1, CA2, Assignment, Mid-Term) with auto-calculation of total and grade based on grading policy.
- FR-EX-004: Results are published by the Exam Controller. Until published, marks are invisible to students.
- FR-EX-005: Students can view published marks, grade, and SGPA/CGPA (if implemented).
- FR-EX-006: Re-evaluation or re-checking requests are submitted online by students within a configurable number of days after result publication.

3.7 Notice Board and Communication

- FR-NOT-001: Admin, HOD, and designated staff can post notices with: title, body (rich text), attachments (PDF/image), target audience (all, specific dept, specific batch, specific role), and visibility dates.
- FR-NOT-002: Students see only notices relevant to their role and department. Notices are sorted by date (newest first) with an unread indicator.
- FR-NOT-003: Critical notices (marked by poster) trigger push notifications (in-app) and SMS to the target group.
- FR-NOT-004: SMS Inbox shows all SMS communications sent to the logged-in student.
- FR-NOT-005: Discussion Forum: students and faculty can post questions in department channels. Faculty moderate posts. Spam/abuse reports go to HOD.

3.8 Grievance Management

- FR-GRIEV-001: Students submit grievances via a typed form with category (academic, financial, administrative, hostel, library, other), description (max 1000 characters), and optional attachments.
- FR-GRIEV-002: Grievances are routed based on category: academic → HOD, financial → Account Officer, hostel → Hostel Warden, library → Librarian, administrative → Admin.
- FR-GRIEV-003: Each grievance has a unique ticket ID, submission timestamp, and status (open / in review / resolved / escalated).
- FR-GRIEV-004: Assigned officer must respond within SLA (configurable, e.g., 5 working days). Overdue grievances escalate to the Principal automatically.
- FR-GRIEV-005: Resolution notes are recorded. Student receives SMS/notification when status changes.

3.9 Administrative and Reporting Module

- FR-RPT-001: MIS Dashboard (Principal view): total students enrolled, active faculty, fee collection today/month/year, attendance summary, pending grievances, library fine outstanding.

- FR-RPT-002: Department-wise reports: enrollment, attendance, marks averages, fee defaulters.
- FR-RPT-003: All reports are exportable as PDF and Excel. Scheduled reports can be emailed daily/weekly to Principal and HODs.
- FR-RPT-004: Audit Log Viewer: Super Admin and Principal can search audit logs by user, action, date range, and resource type.
- FR-RPT-005: System health dashboard: active users online, failed login attempts today, payment gateway status, last DB backup timestamp, disk space.

3.10 Placement and Corporate Relations

- FR-PLACE-001: Placement Officer (staff role) posts job opportunities with company name, role, eligibility criteria (CGPA, branch, year), last date, and application link or in-system form.
- FR-PLACE-002: Students matching eligibility criteria are notified. Students apply from within the system. Officer sees applicant list.
- FR-PLACE-003: Placement results (offer letters uploaded) are recorded per student. Placement statistics visible to Principal.
- FR-PLACE-004: Training calendar: upcoming skill-building sessions, webinars, and workshops posted with registration.

3.11 System Configuration Module (Super Admin)

- FR-CFG-001: Academic year management: create, activate, and archive academic years.
- FR-CFG-002: Registration window management: open/close semester registration, hostel registration, bus registration with exact datetime.
- FR-CFG-003: Fine rate management: library fine per day, late fee penalty rate, all configurable without code deployment.
- FR-CFG-004: Holiday calendar: mark holidays and restricted days that affect attendance calculation and fine computation.
- FR-CFG-005: SMS template management: edit notification templates for all automated SMS messages.
- FR-CFG-006: Payment gateway configuration: switch between gateway providers, update API keys, set test/live mode.
- FR-CFG-007: Backup management: trigger manual database backup, view backup history, configure automated backup schedule and retention.

4. Non-Functional Requirements

4.1 Performance

- NFR-PERF-001: Page load time must be under 2 seconds for 95% of requests under normal load (500 concurrent users).
- NFR-PERF-002: Payment processing response time must be under 5 seconds (gateway-dependent).
- NFR-PERF-003: Database queries must execute in under 200ms for 99% of common operations (timetable view, attendance mark, fee view).
- NFR-PERF-004: Bulk operations (CSV import of 1000 students) must complete in under 60 seconds.
- NFR-PERF-005: Report generation for academic year data must complete in under 10 seconds.

4.2 Security

- NFR-SEC-001: All communication must use TLS 1.2 or higher. TLS 1.0/1.1 must be disabled.
- NFR-SEC-002: SQL injection prevention via parameterized queries only. ORM or prepared statements required throughout.
- NFR-SEC-003: XSS prevention via output encoding and Content-Security-Policy headers.
- NFR-SEC-004: CSRF protection via SameSite cookies and CSRF tokens on all state-changing operations.
- NFR-SEC-005: Rate limiting on authentication endpoints (max 10 requests per minute per IP).
- NFR-SEC-006: Sensitive data (Aadhaar, contact numbers) must be encrypted at rest using AES-256.
- NFR-SEC-007: Security audit trail must be immutable — audit log entries cannot be deleted or modified by any user including Super Admin via the application.
- NFR-SEC-008: Dependency vulnerability scanning must be run before each production deployment (npm audit, pip-audit).

4.3 Reliability and Availability

- NFR-REL-001: System uptime target of 99.5% monthly (excluding planned maintenance windows).
- NFR-REL-002: Planned maintenance must be preceded by at minimum 24 hours advance notice via the notice board.
- NFR-REL-003: Automated database backup daily at 2:00 AM with 30-day retention. Weekly backups retained for 6 months.
- NFR-REL-004: In case of payment gateway failure, transaction state must be recoverable without double-debit via idempotency keys.

4.4 Maintainability and Debuggability

- NFR-MAINT-001: All application errors must be logged with: timestamp, user_id, request URL, request parameters (sensitive values masked), stack trace, and unique error_id shown to the user.
- NFR-MAINT-002: Error codes must be documented in a developer runbook. When a student or staff reports an issue, they cite the error_id, allowing any developer (even unfamiliar with the codebase) to trace it.
- NFR-MAINT-003: Environment configuration (DB URL, API keys, SMTP) via environment variables, never hardcoded.
- NFR-MAINT-004: The codebase must include inline comments for all business logic functions. Complex SQL queries must have explanatory comments.
- NFR-MAINT-005: API must be versioned (/api/v1/) to allow backward-compatible upgrades.

- NFR-MAINT-006: All database schema changes must be applied via migration scripts (Flyway / Liquibase / custom numbered migrations), never direct ALTER TABLE in production without a migration file.
- NFR-MAINT-007: A CHANGELOG.md must be maintained with every release, listing changes, bug fixes, and migration steps.

4.5 Usability

- NFR-USE-001: The student-facing interface must require no training. All primary actions (fee payment, timetable view, attendance check) must be reachable within 3 clicks from the dashboard.
- NFR-USE-002: All forms must provide inline validation with clear, human-readable error messages (not generic “Error 422”).
- NFR-USE-003: UI must be fully responsive across desktop (1280px+), tablet (768px), and mobile (360px+).
- NFR-USE-004: Loading states (spinners, skeleton screens) must be shown for all async operations lasting more than 300ms.
- NFR-USE-005: Destructive actions (delete, deactivate) must require a confirmation dialog with the action described explicitly.

4.6 Scalability

- NFR-SCALE-001: The architecture must support horizontal scaling of the application tier (stateless API servers behind a load balancer).
- NFR-SCALE-002: Database connection pooling (PgBouncer or equivalent) must be used to prevent connection exhaustion.
- NFR-SCALE-003: Static assets (CSS, JS, images) must be served via CDN or efficient caching with cache-control headers.

5. System Constraints and Business Rules

5.1 Date and Time Rules

Rule ID	Business Rule	Enforcement
BR-DATE-001	Registration window open/close times are in IST (UTC+5:30). Server enforces this.	Server-side only
BR-DATE-002	Library fine starts accruing from the day after due date (not the due date itself).	Database trigger
BR-DATE-003	Attendance marked after class end time + 30 min buffer is flagged as late entry and requires reason.	API validation
BR-DATE-004	Fee late payment fine starts from the day after the due date.	Database trigger + cron
BR-DATE-005	Holiday calendar dates suppress fine accrual and attendance requirements.	Business logic layer
BR-DATE-006	Semester registration confirmation email/SMS includes exact timestamp for legal proof.	Application layer

5.2 Financial Business Rules

Rule ID	Business Rule
BR-FIN-001	No fee receipt is issued until payment is confirmed by the gateway webhook callback, not just the redirect.
BR-FIN-002	Partial payments are allowed only if explicitly enabled per fee head by Account Officer.
BR-FIN-003	Fee concessions are applied before fine calculation (fine = (original - concession - paid) * daily_rate * overdue_days).
BR-FIN-004	If a payment fails after bank debit, the Account Officer is alerted. Student is shown pending status, not failure, until gateway confirms.
BR-FIN-005	Hostel and bus fee, once paid, is non-refundable after room/seat allotment (displayed to student before payment).
BR-FIN-006	All financial report figures reconcile with gateway settlement reports daily.

5.3 Academic Business Rules

- BR-ACAD-001: A student cannot register for more than the maximum allowed credits per semester (configurable per department/year).
- BR-ACAD-002: A course cannot be assigned to a batch if no active teacher is assigned to it for that semester.
- BR-ACAD-003: Two timetable entries cannot occupy the same classroom, teacher, or batch slot simultaneously.
- BR-ACAD-004: Results cannot be published if marks entry is incomplete for any eligible student.
- BR-ACAD-005: A student is eligible for the exam only if attendance in that subject is at or above the configured threshold (default 75%).

6. Database Schema Extension Requirements

The existing database schema (files 01–09) covers timetable and core entities well. The following additional tables and modifications are required to support the full ERP scope:

6.1 New Tables Required

Table Name	Purpose	Key Columns
fee_heads	Defines fee types per dept/year/semester	head_id, name, amount, department_id, academic_year, semester, is_mandatory
fee_concessions	Stores concession rules	concession_id, student_id, head_id, type (percent/flat), value, approved_by
fee_transactions	All payment records	txn_id, student_id, order_id, gateway_txn_id, amount, status, method, receipt_url, timestamp
fee_ledger	Running balance per student per head	ledger_id, student_id, head_id, amount_due, amount_paid, fine_accrued, last_calculated_at
library_books	Book catalog	book_id, isbn, title, author, publisher, edition, year, category, total_copies, available_copies, rack
library_issues	Issue/return tracking	issue_id, book_id, student_id, issued_at, due_date, returned_at, fine_amount, fine_paid
library_fines	Fine ledger per student	fine_id, student_id, issue_id, amount, paid_amount, paid_at, txn_id
hostel_categories	Hostel room types and fees	category_id, name, fee_per_term, total_rooms, available_rooms, ac, sharing_type
hostel_registrations	Hostel booking records	reg_id, student_id, category_id, academic_year, room_number, local_guardian_name, local_guardian_contact, status, txn_id
bus_routes	Bus route definitions	route_id, name, stops (jsonb array with stop name + distance + fee), is_active
bus_registrations	Student bus pass records	reg_id, student_id, route_id, stop_name, fee, pass_valid_from, pass_valid_to, txn_id, qr_code
semester_registrations	Semester registration records	sreg_id, student_id, academic_year, semester, courses (int[]), status, registered_at, approved_by
registration_windows	Configurable open/close windows	window_id, type (semester/hostel/bus), academic_year, opens_at, closes_at, is_active, created_by
attendance	Class-wise attendance	att_id, entry_id, student_id, date, status (P/A/ML/DL), marked_by, marked_at, edited_by, edit_reason
exam_schedules	Exam timetable	exam_id, course_id, exam_type, date, start_time, end_time, max_marks, room

Table Name	Purpose	Key Columns
marks	Student marks per exam component	mark_id, student_id, exam_id, component (CA1/CA2/etc), marks_obtained, entered_by, locked, published
grievances	Student grievance tickets	grievance_id, student_id, category, description, attachments, status, assigned_to, ticket_number, created_at, resolved_at, resolution_notes
notices	Notice board posts	notice_id, title, body, attachments, posted_by, target_role, target_dept, target_batch, visible_from, visible_to, is_critical
placements	Job opportunity postings	placement_id, company, role, description, eligibility_cgpa, eligible_branches, apply_deadline, status
placement_applications	Student applications to placements	app_id, placement_id, student_id, applied_at, status, offer_letter_url
system_config	Key-value system configuration	config_key, config_value, description, last_updated_by, last_updated_at
notifications	In-app notification queue	notif_id, user_id, message, type, is_read, created_at, link
sms_logs	SMS delivery tracking	sms_id, recipient_number, message, provider_msg_id, status, sent_at, delivered_at

6.2 Modifications to Existing Tables

- users: add photo_url, mobile_number (encrypted), otp_hash, otp_expires_at, failed_login_attempts, locked_until, must_change_password
- students: add cgpa, sgpa (latest), placement_status, noc_issued, library_membership_id
- timetable_entries: existing conflict constraints are solid; add is_cancelled (boolean) for class cancellation without deletion
- teacher_absences: add leave_type (medical/casual/duty/exam), attachment_url (proof upload)

7. UI/UX Design Guidelines

The following guidelines ensure the interface looks professionally designed, not AI-generated, and appeals to the student demographic (18–22 years old) while meeting institutional credibility standards.

7.1 Design Language

- Adopt a clean, modern flat design with subtle shadows and rounded corners (border-radius: 8–16px). Avoid skeuomorphism and excessive gradients.
- Color palette: Primary — deep navy (#1A3A5C) for institutional trust; Accent — electric blue (#2563EB) for CTAs; Success — emerald (#059669); Warning — amber (#D97706); Danger — rose (#E11D48); Neutral grays for backgrounds.
- Typography: Inter or Poppins as the primary sans-serif. Headers: 700 weight. Body: 400 weight. Minimum font size: 14px for body text, 12px for metadata.
- Icons: Lucide Icons or Phosphor Icons consistently throughout (no mixing icon libraries).
- Motion: Subtle transitions (150–200ms ease-in-out) on hover and state changes. No janky or excessive animations.

7.2 Layout Principles

- Student dashboard: card-based grid with priority information above the fold (attendance %, pending fees, upcoming classes, unread notices).
- Admin interfaces: data-dense sidebar navigation with collapsible sections. Tables with sorting, filtering, and pagination.
- Mobile: bottom navigation bar for students (Dashboard, Timetable, Attendance, Fees, More). No hamburger menus on mobile for primary actions.
- Consistent 8px grid system. Generous whitespace. No information overload on a single screen.
- Empty states: meaningful illustrations with action prompts (not just “No data found” text).

7.3 Student-Specific UX Requirements

- Onboarding: First login shows a guided tour of the dashboard (3–5 tooltips, skippable).
- Attendance widget: circular progress chart per subject on the dashboard. Color-coded: green (75%+), amber (65–74%), red (below 65%).
- Fee payment: clear visual distinction between paid, pending, and overdue. Amount due highlighted in red with a one-click pay button.
- Timetable: weekly calendar grid view (not a plain list). Today’s classes highlighted. Substitutions clearly labeled.
- Dark mode: implemented and toggleable. Preference saved per user.

7.4 Accessibility

- WCAG 2.1 Level AA compliance as a baseline goal.
- All images have alt text. Color is never the sole indicator of information (always paired with icon or text).
- Form fields have visible labels (not just placeholder text). Error messages linked to their fields via aria-describedby.
- Keyboard navigable throughout. Focus indicators visible.

8. Implementation Roadmap

The system is proposed to be built in three phases, each delivering usable value:

Phase	Duration	Scope	Target Users
Phase 1 — Core ERP	4 months	Auth, Student profiles, Timetable (existing), Attendance, Basic Fee (online payment), Notice board	Students, Faculty, Admin
Phase 2 — Finance & Library	3 months	Full fee module (hostel, bus, fines), Library management, Semester registration with date windows, Marks entry	Account Office, Librarian, Students
Phase 3 — Advanced & MIS	3 months	Grievance, Placement, MIS reports, Audit viewer, SMS notifications, Mobile PWA optimization	Principal, HOD, Placement Officer

8.1 Recommended Tech Stack

Layer	Recommended Technology	Rationale
Backend	Node.js + Express (or Fastify)	Fast development, huge ecosystem, async-friendly for payment webhooks
Frontend	React + Vite + Tailwind CSS	Component reuse across roles, Tailwind for consistent design system without AI-look
Database	PostgreSQL 14+ (existing schema)	Retain existing investment; best for complex queries and RLS
ORM/Query	Drizzle ORM or Knex.js	Type-safe, lightweight; avoids raw SQL injection risks without hiding SQL logic
Authentication	JWT + HttpOnly cookies + bcrypt	Industry standard; secure against XSS (HttpOnly) and CSRF (SameSite)
Payment	Razorpay	Best UPI support in India; webhook-based confirmation
File Storage	MinIO (self-hosted) or AWS S3	For receipts, documents, profile photos, attachments
Email	Nodemailer + SMTP (Google Workspace or SendGrid)	Reliable transactional email
SMS	Fast2SMS or MSG91	Cost-effective Indian SMS gateway with template support
Job Queue	BullMQ + Redis	For scheduled jobs: fine accrual, reminders, report generation
Logging	Winston + daily-rotate-file + ELK or Loki	Structured logs with error_id for developer traceability
Migrations	Flyway or custom numbered JS migrations	Safe, audited schema changes
Hosting	VPS (DigitalOcean / Hetzner) or college server	Full control; VPS recommended for reliability

Layer	Recommended Technology	Rationale
Process Manager	PM2 (development) → Docker + Nginx (production)	Zero-downtime deploys with Docker; PM2 for local testing

8.2 Error Traceability Strategy

Every error in the system must be traceable by any developer, even without knowledge of the original code. The following strategy must be implemented:

1. Every error thrown in the backend is caught by a global error handler.
2. The handler generates a unique error_id (UUID v4), logs the full context (user, route, params, stack trace, timestamp) to the error log file.
3. The frontend displays: “Something went wrong. Error ID: [error_id]. Please contact IT support with this ID.”
4. A developer can grep the log file for the error_id to get the full picture in seconds.
5. Error codes are documented: e.g., ERR-FEE-001 = payment gateway timeout, ERR-AUTH-003 = account locked, so IT staff can diagnose without reading code.

9. Student Admission Onboarding and Credential Provisioning

This section defines the end-to-end flow from the moment a student is admitted to the point they first log into UniCampus ERP. This process is owned by the Admission Cell (a dedicated Staff sub-role).

9.1 Admission Cell Role

A dedicated role ‘ADMISSION_STAFF’ (sub-role under STAFF) has permission only to:

- Create new student records from admission forms
- Bulk import students via structured Excel/CSV template
- Generate and dispatch login credentials
- View onboarding status (credential sent / first login done / password changed)
- Reset credentials for students who lost them before first login

Admission Staff CANNOT access fee records, marks, attendance, or any other module. Least-privilege by design.

9.2 Credential Generation Flow

The following sequence is enforced without exception:

6. Admission Confirmed: HOD or Admin marks admission status = confirmed for a student in the system.
7. Student Record Created: Admission Staff creates the student profile with: full name, roll number, date of birth, department, batch, section, mobile number, and parent email/mobile.
8. Credential Auto-Generation: System generates a username (format: rollnumber@collegecode, e.g., BCA240001@FUGS) and a temporary password (cryptographically random, 10 characters, alphanumeric + special).
9. Password stored as bcrypt hash (work factor 12). The plaintext is NEVER stored anywhere in the database after this step.
10. Delivery: Credentials are sent simultaneously via SMS to the student’s registered mobile AND email to the parent/student email. The SMS template reads: “Welcome to [College Name]. Your UniCampus login: Username: [username] | Temp Password: [password]. Login at [URL]. Change your password on first login. — [College Name]”
11. First Login Enforcement: The must_change_password flag is TRUE. On first login, the system forces a password change screen before any other page is accessible. The new password must meet: min 8 chars, 1 uppercase, 1 number, 1 special character.
12. Onboarding Complete: After password change, the student is shown a one-time welcome tour of the dashboard. The must_change_password flag is set to FALSE.

9.3 Bulk Admission Import

- FR-ADM-001: Admission Staff can download a standardised Excel template with required columns: roll_number, first_name, last_name, date_of_birth, gender, department_code, batch_name, section, mobile_number, parent_email, admission_type, category.
- FR-ADM-002: On upload, the system validates every row before any record is created. Validation errors are returned as a downloadable error report (Excel) with row numbers and error descriptions. Partial imports are NOT allowed — either all rows succeed or none are created.
- FR-ADM-003: After successful import, the system generates credentials for all imported students and queues SMS/email delivery in batches (max 100 SMS/minute to respect gateway rate limits).
- FR-ADM-004: A bulk import report is generated showing: total rows processed, successful creations, failed rows, SMS delivery status per student. This report is downloadable by Admission Staff and Super Admin.
- FR-ADM-005: Duplicate roll numbers are rejected with a clear error. If a student is re-admitted (same roll number, different academic year), the existing account is reactivated rather than creating a duplicate.

9.4 Credential Recovery

- FR-ADM-010: Students can self-recover passwords via OTP sent to their registered mobile number (OTP valid for 10 minutes, 6 digits).
- FR-ADM-011: If the mobile number is wrong or inaccessible, student must physically visit the Admission Cell. Staff verifies identity and triggers a credential reset. This action is audit-logged with staff ID and reason.
- FR-ADM-012: Admin and Admission Staff can see a dashboard of students who have NOT completed first login within 7 days of credential dispatch, enabling follow-up.

10. Data Export and Report Generation

All key data in UniCampus ERP must be exportable. Reports serve different stakeholders — the Principal needs summary MIS, the Account Officer needs fee ledgers, and HODs need batch-level academic reports. Export capability is not an afterthought; it is a first-class feature required for accreditation, audit, and institutional decision-making.

10.1 Export Formats Supported

Format	Use Case	Library
Excel (.xlsx)	Fee ledgers, student lists, attendance registers, marks sheets, placement data	ExcelJS (Node.js)
PDF (.pdf)	Official receipts, ID cards, bonafide certificates, admit cards, fee dues notices, registration confirmations	Puppeteer (HTML-to-PDF) or PDFKit
Word (.docx)	Formal letters, NOC letters, transfer certificates, bulk notices to print	docx (npm package)
CSV (.csv)	Raw data export for external analysis, backup, or import into other systems	Built-in Node.js stream

10.2 Student-Facing Exports

Document	Format	Contents	Trigger
Fee Receipt	PDF	Student name, roll no, fee head, amount, date, transaction ID, receipt number, college seal placeholder	Instant after payment confirmation
Fee Dues Statement	PDF + Excel	All fee heads, amounts, concessions, paid amounts, outstanding balance, fine accrued, as-of date	On-demand from student dashboard
Attendance Report	PDF + Excel	Subject-wise: total classes, present, absent, percentage. Date range filterable	On-demand
Timetable	PDF	Weekly grid layout, batch/section, semester, academic year, effective dates	On-demand
Semester Registration Confirmation	PDF	Registration number, student details, courses registered, timestamp, QR code for verification	After registration approval
Library History	PDF + Excel	All issued books, issue dates, return dates, fines paid	On-demand
Bus Pass	PDF (with QR)	Student name, route, stop, validity, photo, QR code	After bus registration payment
Marks / Result Card	PDF	Subject-wise marks per component, total, grade, SGPA (after result publication)	After Exam Controller publishes
Bonafide Certificate	PDF (letterhead)	Auto-generated with student details, course, year, purpose field	On-demand (may require admin approval)
ID Card	PDF / Image	Photo, name, roll number, department, academic year, QR code, validity	Generated on first login completion

10.3 Admin and Staff Exports

Report Name	Roles	Format	Key Contents
Student Master Report	Admin, Principal, HOD	Excel + PDF	All students: name, roll no, dept, batch, contact, parent, status, CGPA, fee status, library status
Fee Collection Report (Daily)	Account Officer, Principal	Excel + PDF	All transactions for a date: student, amount, head, method, gateway ID, status
Fee Collection Report (Monthly/Annual)	Account Officer, Principal	Excel + PDF	Aggregated by head, dept, payment method; reconciliation summary
Fee Defaulters List	Account Officer, HOD, Principal	Excel + PDF	Students with outstanding dues above threshold; includes contact details for follow-up
Attendance Register	HOD, Teacher, Admin	Excel + PDF	Traditional register format: students as rows, dates as columns, P/A/ML/DL cells
Batch-wise Attendance Summary	HOD, Principal	Excel + PDF	Per subject: class count, average attendance %, below-75% student count
Marks Sheet (Internal)	Exam Controller, HOD	Excel + PDF	Component-wise marks for all students in a batch/subject
Library Stock Report	Librarian, Admin	Excel + PDF	All books: copies, issued, available, lost/damaged
Library Fine Outstanding	Librarian, Account Officer	Excel + PDF	All students with unpaid library fines: amount, overdue days, last issued book
Hostel Occupancy Report	Staff (Warden), Admin	Excel + PDF	Room-wise allotment, student name, contact, local guardian, fee status
Bus Route Report	Staff, Admin	Excel + PDF	Route-wise: student list, stops, fee collected
Grievance Status Report	HOD, Principal, Admin	Excel + PDF	All grievances: ticket ID, category, student, assigned to, status, SLA breached?
Placement Report	Placement Officer, Principal	Excel + PDF	Company-wise offers, student placements, CTC ranges, dept-wise statistics
Audit Log Export	Super Admin	Excel + CSV	Filtered audit events: user, action, resource, timestamp, IP
MIS Summary Dashboard Export	Principal	PDF (formatted)	One-page snapshot: enrollment, attendance, fee collection, grievances, placements

10.4 Export Technical Requirements

- FR-EXP-001: All exports are generated server-side. The frontend receives a download URL or a streaming response. Never generate large reports client-side.
- FR-EXP-002: Reports with more than 5,000 rows are generated asynchronously. The user sees a “Generating…” status and receives an in-app notification + email when the file is ready (download link valid for 24 hours).
- FR-EXP-003: All PDF exports use the college letterhead template (logo, college name, address, NAAC/NBA accreditation badges) with a document-specific header and a footer showing “Generated by UniCampus ERP on [datetime] by [username]”.
- FR-EXP-004: All Excel exports include a metadata sheet (Sheet 1: Report Info) with: report name, generated by, generated at, filters applied, and total records. Data starts from Sheet 2.

- FR-EXP-005: Export actions are logged in audit.access_logs with the report type and filters applied, for compliance.
- FR-EXP-006: Sensitive data in exports (mobile numbers, email) is only fully shown to authorised roles. Lower-privilege roles see masked versions (e.g., 90XXXXX695).
- FR-EXP-007: Bulk student data exports require a reason to be typed before download. This reason is logged. Prevents casual data exfiltration.

11. Testing Strategy

A robust, multi-layer testing strategy is mandatory for a system handling financial transactions, academic records, and sensitive student data. This section defines the complete testing approach, tools, and acceptance criteria. Any developer maintaining this codebase in the future must follow this framework.

11.1 Testing Layers Overview

Layer	What It Tests	Tool	Who Writes It	When Run
Unit Tests	Individual functions, service methods, business logic calculations (fine formula, fee computation)	Jest	Developer (author of the function)	On every git commit (pre-commit hook)
Integration Tests	API endpoints end-to-end: request → middleware → controller → service → DB → response	Jest + Supertest	Developer (author of the endpoint)	On every PR to develop branch
Database Tests	SQL queries, migrations, triggers, RLS policies, stored procedures	Jest + pg (test DB)	Developer	On every PR touching DB files
Contract Tests	API response shape matches frontend expectations (TypeScript types match API JSON)	Zod schema validation	Developer	On every PR
UI Component Tests	React components render correctly, handle props, fire events	Vitest + React Testing Library	Frontend developer	On every PR touching frontend
End-to-End (E2E) Tests	Complete user journeys: login → pay fee → get receipt; mark attendance → student sees update	Playwright	QA / Developer	Before every release
Performance Tests	Response time under load (500 concurrent users), DB query benchmarks	k6	Developer / DevOps	Before each Phase release
Security Tests	OWASP Top 10: SQL injection, XSS, CSRF, broken auth, sensitive data exposure	OWASP ZAP (automated) + manual review	Developer	Before each Phase release
User Acceptance Tests (UAT)	Real users validate real workflows match their expectations	Manual (structured test scripts)	College Staff + Students (pilot group)	Before go-live for each Phase

11.2 Unit Testing Requirements

11.2.1 Coverage Targets

- Service layer (business logic): minimum 80% line coverage
- Utility functions (fine calculation, date helpers, validators): minimum 95% line coverage

- Controllers: minimum 70% line coverage
- Database query functions: minimum 70% line coverage

Coverage below these thresholds blocks merging to develop branch via CI check.

11.2.2 Critical Functions That MUST Have Unit Tests

Function	Test Cases Required
calculateLibraryFine(issueDate, returnDate, dailyRate, holidays)	Returned on time (0 fine), returned 1 day late, returned on holiday, returned 30 days late, holiday falls in overdue period (should not count), null returnDate (book not returned yet)
calculateFeeBalance(feeHeads, concessions, payments, dueDate)	No concession, flat concession, percent concession, partial payment, overpayment, payment before due date (no fine), payment after due date (fine applies)
isRegistrationWindowOpen(windowRecord, currentTimestamp)	Window not yet open, window currently open, window closed, window does not exist, window disabled by admin
validatePasswordStrength(password)	Too short, no uppercase, no number, no special char, all conditions met, very long password
generateCredentials(rollNumber, collegeCode)	Correct format output, no collisions across 1000 calls, no plaintext in return object after hash
checkSemesterRegistrationEligibility(studentId)	All clear (eligible), fee dues block, library fine block, HOD hold block, multiple blocks simultaneously
verifyRazorpayWebhookSignature(payload, signature, secret)	Valid signature, tampered payload, wrong secret, empty payload

11.3 Integration Testing Requirements

11.3.1 Test Database Setup

Integration tests run against a dedicated test database (college_timetable_test_db). The test setup must:

13. Run all migrations on the test DB before the test suite.
14. Seed the test DB with a fixed, deterministic dataset (test fixtures) before each test suite.
15. Truncate all tables and re-seed between test suites (not between individual tests for speed).
16. Never run integration tests against the development or production database.

11.3.2 API Endpoint Test Requirements

Every API endpoint must have tests covering at minimum:

- Happy path: valid request, correct role, returns expected response shape and status code
- Unauthenticated request: returns 401
- Insufficient role: returns 403 (e.g., STUDENT trying to access /api/v1/fee/collection)
- Invalid input: returns 400 with structured error and field-level validation messages
- Not found: returns 404 with error code (e.g., GET /api/v1/students/nonexistent-id)
- Business rule violation: returns 422 with the specific ERR-MODULE-NNN error code (e.g., semester registration when window is closed → ERR-STU-001)

Example test structure for the fee payment endpoint:

```
describe('POST /api/v1/fee/orders', () => {  it('should create Razorpay order for
authenticated student with dues')  it('should return 401 if not authenticated')
it('should return 403 if role is TEACHER')  it('should return 422 if student has no
```

```
dues')  it('should be idempotent: duplicate order for same dues returns existing
order')})
```

11.4 End-to-End (E2E) Testing with Playwright

11.4.1 Critical User Journeys to Automate

Journey ID	User Journey	Roles Involved	Priority
E2E-001	Student logs in → views dashboard → views attendance → sees below-75% alert → logs out	STUDENT	P0
E2E-002	Student pays tuition fee → receives on-screen confirmation → downloads PDF receipt → fee balance updates	STUDENT	P0
E2E-003	Admin opens semester registration window → Student registers → HOD approves → student downloads confirmation	ADMIN + STUDENT + HOD	P0
E2E-004	Teacher marks attendance for a class → Student sees updated attendance percentage within 1 minute	TEACHER + STUDENT	P0
E2E-005	Librarian issues book to student → Book unavailable count decreases → Student views active issue	LIBRARIAN + STUDENT	P1
E2E-006	Student returns book 3 days late → Fine calculated correctly → Student pays fine online → Fine cleared	LIBRARIAN + STUDENT	P1
E2E-007	Student submits grievance → Routed to HOD → HOD resolves → Student notified	STUDENT + HOD	P1
E2E-008	Exam Controller publishes results → Student views marks → Student downloads result card	EXAM_CONTROLLER + STUDENT	P1
E2E-009	Account Officer exports fee defaulters list as Excel → File downloaded with correct data	ACCOUNT_OFFICER	P2
E2E-010	New student admitted → Credentials generated → Student receives SMS → Student logs in → Forced password change → Dashboard visible	ADMISSION_STAFF + STUDENT	P0

11.4.2 E2E Environment

- E2E tests run against a dedicated staging environment with a copy of the test database.
- Razorpay test mode credentials used. No real money ever in E2E tests.
- SMS and email are mocked in the E2E environment (captured, not sent).
- E2E suite must complete in under 15 minutes on CI. Slow tests are parallelized across Playwright workers.

11.5 Performance Testing with k6

Before each Phase release, a load test must be run with the following scenarios:

Scenario	Virtual Users	Duration	Pass Criteria
Student dashboard load (read-heavy)	500 VU	5 minutes	P95 response < 2s, error rate < 0.1%
Concurrent fee payment	50 VU	5 minutes	P95 response < 5s, zero duplicate transactions
Attendance marking (faculty, concurrent)	100 VU	5 minutes	P95 response < 1s, no data loss
Report export (PDF generation)	20 VU	5 minutes	P95 complete < 10s, no server crash
Spike test (exam result release)	1000 VU spike over 30 seconds	2 minutes	No 5xx errors, P95 < 5s under spike

11.6 Security Testing

Test	Method	Pass Criteria
SQL Injection on all input fields	OWASP ZAP automated scan + manual payloads on key endpoints	Zero successful injections; all inputs parameterized
XSS on text inputs (names, notice body, grievance text)	OWASP ZAP + manual browser test	CSP headers prevent execution; output encoding in place
CSRF on state-changing endpoints	Manual: remove CSRF token, replay requests	All POST/PUT/PATCH/DELETE return 403 without valid CSRF token
Broken Authentication: JWT tampering	Modify JWT payload (change role to SUPER_ADMIN), replay	Signature validation rejects tampered tokens
Insecure Direct Object Reference (IDOR)	Student A tries to access Student B's fee records, results	RLS + application-level checks return 403
Rate limiting on login	50 rapid login attempts from one IP	Account locked after 5 failures; IP rate-limited
Sensitive data in logs	Trigger errors with card-like numbers in payload	Logs show [REDACTED] for sensitive fields
Password in API response	Fetch user profile API	password_hash NEVER appears in any API response

11.7 User Acceptance Testing (UAT)

UAT is conducted before go-live for each Phase with a pilot group of real college stakeholders. This is the gate that determines if the system is ready to replace iCampus ERP.

11.7.1 UAT Pilot Group Composition

- 2 students (one from each year if possible)
- 2 faculty members (one experienced, one less tech-savvy)
- 1 account officer staff member
- 1 librarian

- 1 HOD
- 1 administrative staff

11.7.2 UAT Process

17. UAT Test Scripts are prepared — step-by-step instructions written in plain language for each user journey. No technical jargon. Example: “Log in with your credentials. Click on ‘Fees’. Look at the amount shown. Does it match your actual dues? Click Pay. Choose UPI. Complete payment. Did you receive an SMS?”
18. Participants execute test scripts independently on the staging environment. They are observed but not helped (to identify genuine usability issues).
19. Every issue is logged: description, severity (Critical / Major / Minor / Cosmetic), screenshot, tester name, date.
20. Critical issues (system crash, wrong calculation, payment failure) must be fixed before go-live. Major issues have 2 weeks to fix. Minor and cosmetic issues go into the next release backlog.
21. UAT sign-off: each pilot group member signs a UAT acceptance form confirming the system meets their workflow needs. This form is stored in docs/uat/.

11.7.3 UAT Acceptance Criteria (Phase 1 Go-Live)

- 100% of P0 E2E tests pass
- Zero Critical UAT issues open
- Less than 3 Major UAT issues open (with fix committed to within 2 weeks)
- All pilot group members able to complete their primary workflows without developer assistance
- Performance test pass criteria met
- Security scan: zero High or Critical vulnerabilities open

12. Migration from iCampus ERP to UniCampus ERP

The college currently runs iCampus ERP by Digital Dreams Systems. Migrating from a live, production ERP to a new system while the college continues operating is a high-risk operation that requires careful planning. A botched migration can result in lost student records, fee payment gaps, or loss of institutional trust. This section defines the complete migration strategy.

12.1 Migration Principles

- Zero data loss: Every student record, fee transaction, attendance record, and library record from iCampus must exist in UniCampus after migration.
- Zero downtime migration: The college cannot afford a full system downtime. Migration is done in phases while both systems run in parallel.
- Verified before cutover: Data is migrated and cross-checked against iCampus before any cutover. Discrepancies are resolved before the old system is decommissioned.
- Rollback plan: If migration fails at any stage, iCampus remains the source of truth and can be reverted to immediately.
- Audit trail preserved: Historical audit logs from iCampus (in whatever format they export) are archived in a read-only table in UniCampus for compliance.

12.2 Data to Migrate

Data Category	Source in iCampus	Target in UniCampus	Migration Method	Priority
Student master records	iCampus student DB export	students + users tables	ETL script (CSV → validated → insert)	P0 — Day 1
Faculty records	iCampus faculty export	teachers + users tables	ETL script	P0 — Day 1
Department and course catalog	iCampus admin export	departments, courses tables	Manual + ETL	P0 — Day 1
Historical fee transactions	iCampus accounts export	fee_transactions (read-only historical)	ETL + manual verification	P1 — Week 1
Current fee dues (live)	iCampus fee ledger	fee_ledger table	Manual re-entry + cross-check	P0 — Before fee collection starts
Attendance records (current year)	iCampus attendance export	attendance table (historical flag)	ETL script	P1 — Week 2
Library catalog (books)	iCampus library export	library_books table	ETL script	P1 — Week 2
Currently issued books (active)	iCampus library module	library_issues table	Manual re-entry (critical — small number)	P0 — Before library goes live
Timetables (current semester)	iCampus timetable export	timetables + timetable_entries	Manual re-entry by HODs (UI available)	P1 — Before timetable goes live
Historical marks and results	iCampus exam export	marks table (historical flag = TRUE)	ETL script + Exam Controller verification	P2 — Month 2
Hostel allotments (current)	iCampus hostel module	hostel_registrations table	Manual re-entry	P1

Data Category	Source in iCampus	Target in UniCampus	Migration Method	Priority
Bus registrations (current)	iCampus bus module	bus_registrations table	Manual re-entry	P1
User credentials	NOT migrated	Regenerated for all users	Bulk credential generation + SMS dispatch	P0 — Week before go-live

12.3 ETL Script Requirements

ETL (Extract, Transform, Load) scripts convert iCampus data exports (typically CSV or Excel) into UniCampus database records.

- All ETL scripts are idempotent: running them twice produces the same result (no duplicate records).
- ETL scripts validate every record before insert: roll number format, email format, date ranges, foreign key existence.
- Failed records are written to an error log with: original row data, error description, line number. Migration does not stop on individual row failure.
- After ETL completion, a reconciliation report is generated: rows attempted, rows inserted, rows failed, checksum of key fields (total students, total fee amount).
- ETL scripts are version-controlled in database/migration/etl/ and must be reviewed by a second developer before running on production.

12.4 Parallel Running Period

Both iCampus and UniCampus run simultaneously for a period to validate correctness:

Week	Activity
Week 1–2 (Shadow Mode)	UniCampus is deployed but only accessible to the development team and 5 pilot students. All operations still happen in iCampus. UniCampus data is verified against iCampus daily.
Week 3–4 (Pilot Group)	30–50 pilot users (students + 2 faculty + 1 account officer) use UniCampus for read operations (view timetable, attendance, fees). iCampus still handles writes. Feedback collected and critical bugs fixed.
Week 5–6 (Write Pilot)	Pilot group performs write operations in UniCampus (mark attendance, submit grievance, pay a small fee). Same operations manually verified in iCampus for cross-check. No fee collection cutover yet.
Week 7 (Fee Collection Cutover)	Fee collection moves to UniCampus for new payments. iCampus fee module disabled for new entries (read-only). Account Officer verifies daily settlement matches.
Week 8 (Full Cutover)	All modules cut over to UniCampus. iCampus set to read-only for 30 days (students can access historical data). After 30 days, iCampus decommissioned.
Month 3 (Decommission)	iCampus data fully exported and archived in docs/archive/icampus_final_export/. iCampus contract terminated. All DNS/links updated to UniCampus.

12.5 Rollback Plan

- At any point during parallel running (weeks 1–6), rollback = stop using UniCampus. No data is lost in iCampus since it remains the write system.
- After fee collection cutover (week 7+): rollback requires exporting UniCampus fee transactions and manually entering them into iCampus. This is the point of no return for fee data. Requires Account Officer sign-off before week 7 cutover.

- A rollback decision must be made within 4 hours of any critical issue post-cutover. After 4 hours with the issue unresolved, rollback is automatic.
- All rollback scenarios are documented and tested in the staging environment before any live cutover.

12.6 Staff Training Before Cutover

- Mandatory 2-hour training session for each staff role group before they begin using UniCampus in write mode.
- Training materials: role-specific user manuals (PDF), video walkthroughs for top 5 tasks per role, quick reference card (1-page PDF).
- IT helpdesk availability: dedicated support line/channel for the first 30 days post go-live.
- Student communication: official notice 2 weeks before cutover via SMS, email, and notice board. Includes login instructions and FAQ.

13. Project Timeline and Structured Workplan

This timeline is designed for a single developer (Zishan Ahmad) with approximately 20–25 hours per week available (accounting for college schedule). It is aggressive but achievable if the project is treated as a priority. External dependencies (college approval, payment gateway KYC, SMS gateway) must be resolved in parallel with development.

13.1 Pre-Development (Weeks 1–2)

Task	Owner	Deliverable	Due
Get principal approval on SRS document	Zishan (present to principal)	Signed SRS approval page	Week 1
Create GitHub repository with branch structure (main, develop)	Zishan	Repo live with README, .env.example, folder structure	Week 1
Apply for Razorpay test account (KYC takes 3–5 days)	Zishan / College accounts dept	Razorpay test API keys	Week 1
Apply for Fast2SMS API key	Zishan	SMS API key in hand	Week 1
Set up local development environment (Docker Compose: PG + Redis + MinIO)	Zishan	docker-compose up works, all services healthy	Week 1
Run existing SQL files (00–09) on local DB, verify schema	Zishan	All tables created, sample data loaded, queries run	Week 1
Write all numbered migration files (010–032) for new tables	Zishan	32 SQL migration files checked into database/migrations/	Week 2
Set up CI pipeline (GitHub Actions: lint + test + build on every PR)	Zishan	.github/workflows/ci.yml working	Week 2
Write docs/ERROR_CODES.md with all known error codes	Zishan	File committed with initial 30+ error codes	Week 2
Collect iCampus data export samples from admin (for ETL planning)	Zishan + Admin	Sample CSV exports from iCampus for each module	Week 2

13.2 Phase 1 — Foundation (Weeks 3–10, ~8 weeks)

Month 1 (Weeks 3–6) — Backend Core

Week	Tasks	Milestone
Week 3	Backend project init (Express + TypeScript + Drizzle). DB connection, migration runner. Auth module: login endpoint, JWT generation, HttpOnly cookie, bcrypt verify.	Auth API: login works, returns JWT
Week 4	Auth: refresh token, logout (DB invalidation), OTP generation + SMS send, password reset flow, must_change_password enforcement. Role middleware. Error handler global.	Full auth flow works end-to-end
Week 5	Student module: CRUD endpoints (admin creates, student reads own). User profile endpoints. Photo upload to MinIO. Admission Staff role: credential generation, bulk import (Excel parse + validate + insert).	Student CRUD + credential generation

Week	Tasks	Milestone
Week 6	Timetable module: connect existing DB schema, GET endpoints for teacher timetable, batch timetable. Substitution endpoints. Notice module: CRUD, targeting by role/dept.	Timetable + Notice APIs working

Month 2 (Weeks 7–10) — Frontend + Attendance

Week	Tasks	Milestone
Week 7	Frontend init (React + Vite + Tailwind). Tailwind config with custom color tokens. Base UI components: Button, Input, FormField, Modal, Skeleton, Badge, DataTable, ConfirmDialog. Auth pages: Login, ForgotPassword.	Frontend boots, login works
Week 8	Student dashboard: attendance widget (circular chart), quick stats cards, notice board widget, upcoming classes widget. Role-based routing + ProtectedRoute. Sidebar navigation per role.	Student dashboard complete
Week 9	Attendance module backend: mark attendance (teacher), edit window enforcement, percentage calculation. Attendance frontend: teacher marking UI, student subject-wise view, HOD batch view.	Attendance module complete
Week 10	Timetable frontend: weekly grid component, today highlight, substitution labels. Admin user management pages. Integration tests for all Phase 1 API endpoints. Phase 1 UAT preparation.	Phase 1 feature-complete

Phase 1 Milestone: UAT with pilot group. Fix critical/major issues. Deploy to staging server.

Phase 1 Deliverables Checklist

- Login, OTP reset, role-based access working for all 10 roles
- Student profile view and edit
- New student admission + credential SMS dispatch
- Bulk student import with error report
- Weekly timetable view for students and teachers
- Attendance marking by faculty + student view with percentage
- Notice board with targeting + student inbox
- All API endpoints have integration tests (70%+ coverage)
- E2E tests E2E-001, E2E-004, E2E-010 passing

13.3 Phase 2 — Finance and Library (Weeks 11–18, ~8 weeks)

Week	Tasks	Milestone
Week 11	Fee module: fee_heads CRUD (admin), fee_ledger computation, concession management. Fee balance API for student.	Fee structure management works
Week 12	Razorpay integration: create order, verify webhook signature, idempotency check, update ledger on success. Receipt PDF generation (Puppeteer).	Online fee payment works end-to-end
Week 13	Offline payment recording (Account Officer). Fee collection report (daily + monthly). Defaulters list export (Excel + PDF).	Fee reporting complete
Week 14	Registration windows: system_config-driven open/close. Semester registration: eligibility check, course selection, PDF confirmation. Hostel registration: category selection + payment.	Semester + Hostel registration

Week	Tasks	Milestone
Week 15	Bus registration: route config, stop selection, fee, QR pass PDF. Library backend: book catalog CRUD, issue/return flow, fine calculation (BullMQ daily job).	Bus + Library backend
Week 16	Library fine payment (online + offline). Library frontend: catalog search, issue/return for librarian, student view. Fine balance on student dashboard.	Library module complete
Week 17	Fee frontend: student fee breakdown page, payment flow, receipt download. Account Officer dashboard: daily collection, pending transactions, reconciliation.	Fee frontend complete
Week 18	Phase 2 integration tests + E2E tests (E2E-002, E2E-003, E2E-005, E2E-006). Performance test run. Phase 2 UAT.	Phase 2 feature-complete

Phase 2 Deliverables Checklist

- Online fee payment via Razorpay (UPI, card, netbanking) with webhook-confirmed receipts
- Offline payment recording by Account Officer
- Fee concessions and late fine auto-calculation
- Semester, hostel, and bus registration with date-window controls
- Library issue/return with automated daily fine accrual job
- Library fine payment (online + offline)
- All exports: fee receipt PDF, dues statement Excel, collection report, defaulters list
- Student export: fee dues statement in Excel and PDF

13.4 Phase 3 — Advanced Modules (Weeks 19–26, ~8 weeks)

Week	Tasks	Milestone
Week 19	Exam module: schedule CRUD, eligibility check (attendance threshold). Marks entry: faculty entry window, component CRUD, auto-total.	Marks entry works
Week 20	Marks lock/publish (Exam Controller). Result card PDF. Re-evaluation request flow. Student result view.	Exam module complete
Week 21	Grievance module: ticket creation, category routing, SLA timer (BullMQ), escalation to Principal on breach. Resolution flow.	Grievance module complete
Week 22	Placement module: job posting (Placement Officer), student eligibility filter, application submission, offer letter upload. Placement statistics for Principal.	Placement module complete
Week 23	MIS Dashboard (Principal): real-time stats, charts (Recharts), all-module summaries. Bulk export: student master Excel, MIS PDF snapshot.	MIS dashboard complete
Week 24	System Config admin panel: all configurable values (fine rates, registration windows, thresholds, holidays, SMS templates). Audit Log Viewer (searchable, exportable).	Admin config + audit viewer
Week 25	Security audit (OWASP ZAP), penetration test review, fix all High/Critical issues. Final performance test (k6). Production environment setup (Nginx + SSL + PM2).	Security cleared
Week 26	Full E2E test run (all P0 + P1 journeys). Final UAT with all stakeholder groups. UAT sign-off collection. Migration ETL scripts written + tested on staging.	Ready for go-live

13.5 Go-Live and Migration (Weeks 27–32)

Week	Activity
Week 27–28	Shadow mode: UniCampus deployed, pilot group read-only access. ETL scripts run for P0 data (students, faculty, fee dues). Cross-verify against iCampus.
Week 29–30	Pilot write access: 50 users perform real operations. Staff training sessions. All critical/major issues from UAT resolved and re-tested.
Week 31	Fee collection cutover: Razorpay live mode activated. Account Officer sign-off. Daily reconciliation for 1 week.
Week 32	Full cutover: all modules live. iCampus set to read-only. Official announcement to all students and staff. Dedicated IT support active.

13.6 Post Go-Live (Month 9–12)

- Month 9: Monitor system health daily. Fix bugs from production usage. Collect feature requests.
- Month 10: iCampus decommissioned. Historical data archived. ETL of remaining historical records (old marks, old attendance) completed.
- Month 11: Minor improvements release based on user feedback. Documentation updated.
- Month 12: Annual review with Principal. Roadmap for Year 2 features (mobile app, DigiLocker integration, automated timetable generation, AI-based attendance anomaly detection).

13.7 External Dependency Timeline

Dependency	Required By	Action Required	Lead Time
Razorpay merchant account (live)	Week 31 (fee cutover)	College accounts dept submits KYC documents to Razorpay	10–15 business days
SMS gateway (Fast2SMS) credits + DLT registration	Week 3 (any SMS feature)	Register SMS templates with TRAI DLT portal (mandatory for India)	7–14 business days
VPS / hosting server	Week 10 (staging)	Purchase VPS (Hetzner CX31 or DigitalOcean 4GB droplet recommended)	1 day
Domain name + SSL	Week 10 (staging)	Register domain (unicampuserp.college.edu.in preferred), get free Let’s Encrypt SSL	1–3 days
College letterhead digital template (PNG/SVG)	Week 12 (PDF exports)	Request from college administration	3–5 days
iCampus data export	Week 26 (ETL scripts)	Request complete data export from Digital Dreams Systems. May require formal notice.	14–30 days — start early
Principal formal approval	Week 1	Present SRS. Requires principal’s office meeting.	3–5 days after meeting

14. Additional Features and Gap Analysis

The following features were identified as missing from the initial scope but are important for a complete college ERP. They are prioritised and assigned to appropriate phases.

14.1 Features Added to Scope

Feature	Why Important	Phase
Admission Cell role + credential provisioning workflow	First login experience is the student's first impression of the new system. Poor onboarding destroys trust.	Phase 1
Bulk data export (Excel/PDF/DOCX) for all modules	Required for NAAC accreditation, internal audits, fee reconciliation, and daily college operations.	Phase 1–2
DLT-compliant SMS templates (TRAI mandate)	All Indian SMS providers require pre-registered templates. Without this, SMS will be blocked.	Pre-dev
Holiday calendar in System Config	Holidays must suppress fine accrual and attendance requirements. Missing this causes wrong calculations.	Phase 2
Bonafide and NOC certificate generation	Students constantly need these for bank accounts, visa, other colleges. Currently manual.	Phase 2
Student academic hold by HOD	HOD needs ability to block specific students from registration pending dues/disciplinary action.	Phase 2
Condonation request for attendance	Medical leave with supporting document. Currently no formal workflow in most college systems.	Phase 2
Timetable cancellation (is_cancelled flag)	Faculty needs to cancel a class without deleting the permanent timetable entry.	Phase 1
In-app notification system	Real-time alerts for result publication, notice, payment confirmation, grievance update.	Phase 2
QR code verification for bus pass and registration confirmation	Wardens / transport staff scan QR to verify validity without logging in.	Phase 2
Student academic history view (all semesters)	Students need to see cumulative academic records, not just current semester.	Phase 3
CGPA / SGPA calculation	Automatic computation from published marks. Critical for placement eligibility filtering.	Phase 3
Re-evaluation request workflow	Missing from the original scope. Students must be able to formally request re-checking.	Phase 3
Audit log immutability enforcement	Original schema allowed deletions. Must enforce INSERT-only via DB trigger or separate append-only table.	Phase 1 (DB level)
Rate limiting and DDOS protection at Nginx level	Without this, the system is vulnerable to credential stuffing and service disruption.	Pre go-live
Automated daily database backup with restoration test	Data loss due to server failure without backups would be catastrophic and irrecoverable.	Phase 1 (infrastructure)

Feature	Why Important	Phase
Production error alerting (email on spike)	Without monitoring, production bugs can go unnoticed for days. Critical for a system the college depends on.	Phase 1 (infrastructure)

14.2 Features Deliberately Out of Scope (Phase 1–3)

- LMS / lecture content delivery (video, slides hosting) — dedicated platforms like Google Classroom or Moodle do this better.
- Payroll and HR management — requires separate HR system integration.
- Mobile native app (Android/iOS) — responsive web is sufficient for Phase 1–3. PWA in Phase 3 if needed.
- AI-based timetable auto-generation — complex constraint satisfaction problem; out of scope for v1.
- DigiLocker integration — useful for digital document verification; planned for Year 2.
- Biometric attendance integration — requires hardware procurement; faculty-marked digital attendance is sufficient for Phase 1.
- Multi-college / multi-campus support — architecture is designed to be extendable but this is not needed for Phase 1.

15. Approval and Sign-off

This document requires review and approval from the following stakeholders before development begins:

Name / Role	Department	Signature	Date
Principal / Director	Administration		
Head of IT / System Admin	IT Department		
Account Officer	Accounts		
Head Librarian	Library		
HOD Computer Science	CSE Department		
Student Representative	Student Council		
Developer (Zishan Ahmad)	Development		

Approved version of this document must be version-controlled and stored alongside the source code repository.

Document prepared by: Zishan Ahmad | BCA Student, IIT Madras Data Science | United Institute of Management (FUGS)

Date: June 23, 2026 | Contact: Available via college system post-deployment